MICROBIT. УРОК №5. DINO DASH: ФИЗИКА В ИГРЕ.

Здравствуйте, юные разработчики!

Рад снова всех видеть. Подключайте micro:bit к геймпаду, и компьютеру, открывайте редактор. Пока он загружается, давайте быстро вспомним, что мы уже умеем — это пригодится прямо сейчас.

Блиц-опрос!

- Зачем мы делим проект на несколько файлов (модулей), а не пишем всё в одном main.py?
- Как мы делали «реальное время» в игре, чтобы не зависеть от больших sleep()?
- Что такое «состояние» игры? Пример из прошлого урока.
- Почему в играх полезно иметь переменную-флаг?
- Что важнее в мини-играх на micro:bit: идеальная графика или понятные правила? Почему?

Отлично! Сегодня у нас новая миссия: мы сделаем бесконечный раннер. Это такой жанр игр, где герой бежит вперёд сам, а игрок успевает реагировать на препятствия.

> ## Историческая справка!
>
> Раннеры любят за простоту: правила можно объяснить одной фразой («прыгай через препятствия»), но рекорд всегда хочется побить. На телефонах такие игры стали популярны особенно сильно - ведь одна кнопка управления идеально подходит для коротких сессий.

Наша версия будет называться **Dino Dash**. Динозавр бежит слева, кактус появляется справа и «едет» навстречу. Когда вы нажимаете кнопку — динозавр прыгает. Если прыгнули поздно — столкновение, Game Over, показываем счёт и начинаем заново.

В этом курсе мы учимся писать код «как взрослые программисты»: разделяем проект на модули. И в нынешнем проекте снова будет три файла:

- **gamepad.py** – драйвер нащего геймпада.
- **dino.py** – версия модуля управления персонажем. Напомним, что основное управление персонажем будет выделено в отдельный модуль. Однако, так как от проекта к проекту код будет меняться, перед созданием основной части, мы будем подробно разбирать данный модуль.
- **main.py** — главный файл вашего проекта игры.

Представьте, что **dino.py** — это маленький «робот-динозавр», который умеет делать два дела: двигаться по вертикали (прыгать/падать) и рисовать себя на экране. Главный файл **main.py** будет только отдавать команды: «прыгни», «обнови физику», «нарисуй».

Мы храним состояние персонажа в **переменных**. Прыжок — это не перенос на другую клетку, а изменение скорости. Гравитация — это постоянное «ускорение вниз».

Интересный факт!

Во многих движках гравитация - это просто число, которое каждый кадр добавляется к скорости. То же самое вы делаете сегодня, только на экране 5×5.

```
# dino.py - модуль персонажа для Dino Dash
from microbit import *

# Состояние персонажа
_y = 4      # 4 - это «земля» (нижняя строка)
_vy = 0     # вертикальная скорость

def reset():
    # Возвращаем динозавра в начальное состояние
    global _y, _vy
    _y = 4
    _vy = 0

def on_ground():
    # Стоим ли мы на земле?
    return _y >= 4

def jump():
    # Прыжок только если стоим на земле
    global _vy
    if on_ground():
        _vy = -2   # стартовый импульс вверх

def update_physics():
    # Один шаг физики: скорость, гравитация, приземление
    global _y, _vy

    # 1) двигаемся по скорости
    _y += _vy

    # 2) «гравитация» тянет вниз: скорость падения растёт
    _vy += 1

    # 3) приземление
    if _y >= 4:
        _y = 4
        _vy = 0
```

```
    # 4) защита от выхода за верх (чтобы не исчезать)
    if _y < 1:
        _y = 1

def cells():
    # Какие клетки на экране занимает динозавр
    return [(1, _y), (1, _y - 1)]

def draw():
    # Рисуем динозавра (2 пикселя)
    display.set_pixel(1, _y, 9)       # ноги
    display.set_pixel(1, _y - 1, 5)   # тело/голова
```

Рассмотрим подробнее:

Переменные **_y** и **_vy** — это память персонажа. Если бы мы их не хранили, динозавр «забывал бы», где он находится, и игра превращалась бы в хаос.

jump() не меняет _y напрямую. Он задаёт скорость вверх (_vy = -2). Дальше update_physics() будет использовать эту скорость, и движение получится плавным.

update_physics() делает три ключевых вещи в правильном порядке: сначала двигает по скорости, потом добавляет гравитацию, потом проверяет приземление.

cells() — «секретный помощник» для столкновений. Он возвращает список клеток, которые занимает динозавр. main.py не трогает внутренние переменные, он работает только с результатом cells().

> ***Открытый вопрос:*** *почему мы делаем динозавра из двух пикселей, а не из одного? Как это влияет на столкновения и на ощущение персонажа?*

Теперь самое интересное: приступаем к созданию основного файла игры! Мы будем двигаться маленькими шагами. После каждого шага — запускаем программу и проверяем, что всё работает.

Шаг 1. Каркас проекта и переменные

Откройте main.py и добавьте импорт модулей. Мы сделаем игру простой: на экране будет максимум один кактус за раз. Так будет легче отладить проект

```
from microbit import *
import random
import gamepad
import dino

# Один кактус: None (нет) или список [x, height]
# height = 1 (маленький), 2 (высокий)
cactus = None

score = 0

# Тайминги (миллисекунды)
FRAME_MS = 60          # как часто обновляем физику и рис
MOVE_MS = 260          # как часто двигаем кактус влево
SPAWN_MIN = 450        # минимум до появления нового какт
SPAWN_MAX = 900        # максимум до появления нового как

last_frame = running_time()
last_move = running_time()
next_spawn = running_time() + random.randint(SPAWN_MIN, S
```

> ***Подсказка!*** *FRAME_MS отвечает за плавность. MOVE_MS — за скорость мира. Если сделать FRAME_MS большим, прыжок будет «дёрганым». Если сделать MOVE_MS маленьким — игра станет слишком быстрой.*

> ### Справка!
> Python не имеет представления о таком объекте в программировании, как константа – неизменяемая переменная. Поэтому среди программистов принято обозначать в языке Python такие неизменяемые переменные верхним регистром (заглавные буквы).

Шаг 2. Рисуем землю и кактус

Сначала научимся рисовать: земля — тонкая линия, чтобы было понятно, где низ. Кактус — один или два пикселя в зависимости от высоты.

```
def draw_ground():
    # Земля - слабая линия (яркость 2), чтобы не мешала
    for x in range(5):
        display.set_pixel(x, 4, 2)

def draw_cactus():
    if cactus is None:
        return

    x, h = cactus[0], cactus[1]
    if 0 <= x <= 4:
        display.set_pixel(x, 4, 9)   # основание
        if h == 2:
            display.set_pixel(x, 3, 9)  # верхушка
```

Проверка

Пока мы ещё не сделали главный цикл, но вы уже можете проверить рисование: временно поставьте cactus = [4, 2] в начале и внутри цикла рисуйте кадры. Если увидели кактус справа — значит рисование работает.

Шаг 3. Появление кактуса

Теперь научим игру создавать препятствие. Идея простая: если кактуса нет и наступило время next_spawn — создаём новый справа (x = 4).

```
def try_spawn(now):
global cactus, next_spawn

# Если кактус уже есть - ничего не делаем
if cactus is not None:
    return

# Если пришло время - создаём новый кактус
if now >= next_spawn:
    cactus = [4, random.choice([1, 2])]
    next_spawn = now + random.randint(SPAWN_MIN, SPAWN_MA
```

Интересный факт!

Случайная пауза перед препятствиями делает игру «живой». Если препятствия идут строго по расписанию, игрок быстро начинает угадывать ритм и становится скучно.

Шаг 4. Движение мира и очки

Раннер устроен так: кажется, что динозавр бежит вперёд, но на самом деле мы двигаем препятствия влево. Если кактус ушёл за экран — игрок молодец, прибавляем очко.

```
def move_world():
global cactus, score, MOVE_MS

if cactus is None:
    return
```

```
cactus[0] -= 1  # шаг влево

# Если кактус ушёл за экран - начисляем очко и убираем ег
if cactus[0] < 0:
    score += 1
    cactus = None

    # Чуть-чуть ускоряем игру каждые 3 очка (но не бескон
    if score % 3 == 0 and MOVE_MS > 140:
        MOVE_MS -= 10
```

Самостоятельная работа (2 минуты)

Сделайте игру немного добрее: измените ускорение так, чтобы скорость увеличивалась реже. Например, каждые 5 очков, а не каждые 3.

Ответ (пример):

```
if score % 5 == 0 and MOVE_MS > 140:
MOVE_MS -= 10
```

Шаг 5. Столкновение

Самое важное правило игры: если динозавр и кактус оказались в одной клетке — Game Over. Мы не сравниваем «картинки», мы сравниваем координаты клеток.

```
def is_collision():
if cactus is None:
    return False

d_cells = dino.cells()

x, h = cactus[0], cactus[1]
c_cells = [(x, 4)]
```

```
if h == 2:
    c_cells.append((x, 3))

for cell in c_cells:
    if cell in d_cells:
        return True
return False
```

Шаг 6. Главный цикл игры

Теперь собираем всё вместе. Главный цикл — это сердце игры. Он постоянно:

- читает кнопку прыжка,
- спавнит кактус,
- двигает мир по таймеру,
- обновляет физику и рисует кадр,
- проверяет столкновение.

```
while True:
now = running_time()

# 1) Управление: прыжок по зелёной кнопке
if gamepad.button_green():
    dino.jump()

# 2) Появление препятствий
try_spawn(now)

# 3) Движение мира по таймеру
if now - last_move >= MOVE_MS:
    last_move = now
    move_world()

# 4) Обновление кадра по таймеру
if now - last_frame >= FRAME_MS:
    last_frame = now

    dino.update_physics()

    display.clear()
```

```
        draw_ground()
        draw_cactus()
        dino.draw()

        # 5) Столкновение
        if is_collision():
            gamepad.vibrate(220)
            display.show(Image.SKULL)
            sleep(700)
            display.scroll("SCORE: " + str(score))

            # Перезапуск
            cactus = None
            score = 0
            MOVE_MS = 260
            dino.reset()
            next_spawn = running_time() + random.randint(SPAW
            last_frame = running_time()
            last_move = running_time()

    sleep(5)
```

Практическая часть: тестирование

Запустите игру и проверьте:
1) Динозавр прыгает только с земли (в воздухе повторный прыжок не должен срабатывать).
2) Кактус появляется через случайное время.
3) Когда вы перепрыгнули кактус — счёт увеличивается.
4) При столкновении — вибрация, череп и показ счёта, затем игра начинается заново.

Частые ошибки и быстрые решения

1) Динозавр не прыгает

Проверьте, что вы импортировали dino и вызываете dino.jump() по зелёной кнопке. Также убедитесь, что

в dino.py есть функция on_ground(), иначе прыжок может работать неправильно.

2) Кактус не появляется

Проверьте, что try_spawn(now) вызывается в цикле. Если забыть этот вызов, игра будет пустой. Также проверьте, что next_spawn вычисляется в начале.

3) Счёт не растёт

Очко добавляется, когда кактус уходит за экран (x < 0). Проверьте, что вы реально уменьшаете cactus[0] в move_world().

4) Игра слишком сложная

Увеличьте MOVE_MS (например, до 300) и увеличьте SPAWN_MIN/SPAWN_MAX. Чем больше пауза между препятствиями — тем легче играть.

Если вы справились быстрее, выберите одно улучшение. Делайте только одно — так вы успеете протестировать и не запутаетесь.

- Двойной прыжок: разрешить второй прыжок в воздухе только один раз (нужна переменная-счётчик прыжков).
- Очки за время: добавлять +1 каждые N секунд, даже если кактус не прошёл (сложнее, но интересно).
- Второй тип препятствия: «камень» высотой 1, который появляется чаще, чем высокий кактус.
- Экран рекорда: хранить best_score и показывать его после проигрыша.

Сегодня вы **собрали полноценную игру** и сделали важный шаг к настоящему геймдеву на MicroPython. Вы узнали, как из простых чисел (положение, скорость, таймеры) рождается «живая» механика.

Задание

Раздели элементы проекта на то, что относится к главному файлу (main.py), и то, что относится к модулю игры.с

Главный фай ОК Модуль игры